

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 27

СОЗДАНИЕ И НАСТРОЙКА ПОДКЛЮЧЕНИЯ К БАЗЕ ДАННЫХ POSTGRESQL В DJANGO

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № _____ от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание и настройка подключения к базе данных PostgreSQL в Django

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Создание и настройка подключения к базе данных PostgreSQL в Django

По умолчанию Django в качестве базы данных использует SQLite. Она очень проста в использовании и не требует запущенного сервера. Все файлы базы данных могут легко переноситься с одного компьютера на другой. Однако при необходимости мы можем использовать в Django большинство распространенных СУБД (рисунок 1).

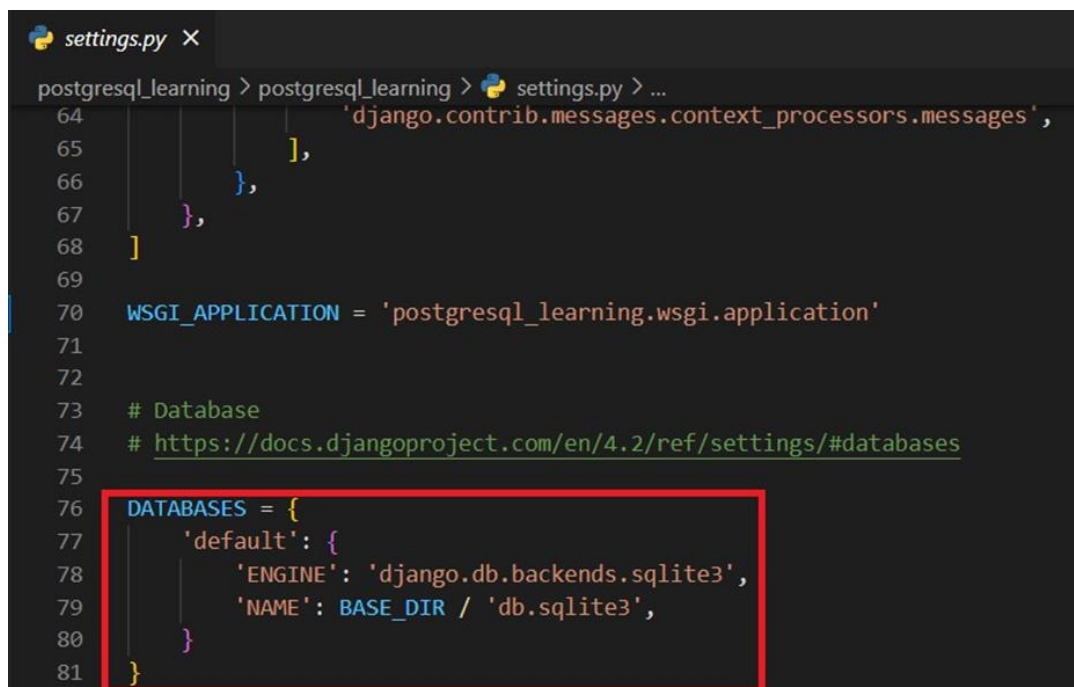


Рисунок 1 - Настройка базы данных по умолчанию в файле settings.py

Поддерживаемые СУБД:

- PostgreSQL (psycopg2);
- MySQL (mysql-python);
- Oracle (cx_Oracle);
- Microsoft SQL Server (mssql-django).

Чтобы подключить PostgreSQL, его нужно для начала установить.

- переходим на сайт <https://www.postgresql.org/download/> и скачиваем PostgreSQL для вашей ОС;
- далее открываем установщик (рисунок 2).

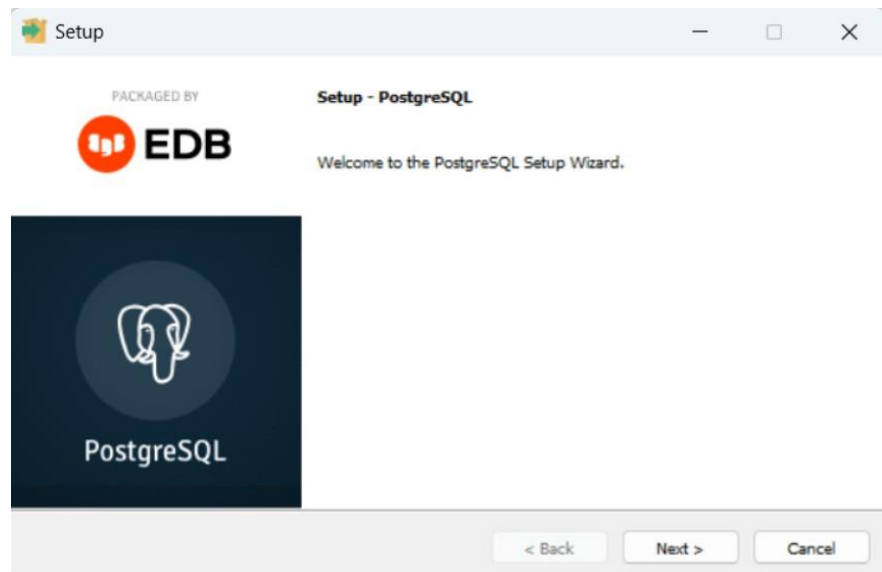


Рисунок 2 - Установка PostgreSQL

- выбираете директорию для установки СУБД;
- выбираете компоненты, идущие с СУБД (рисунок 3):

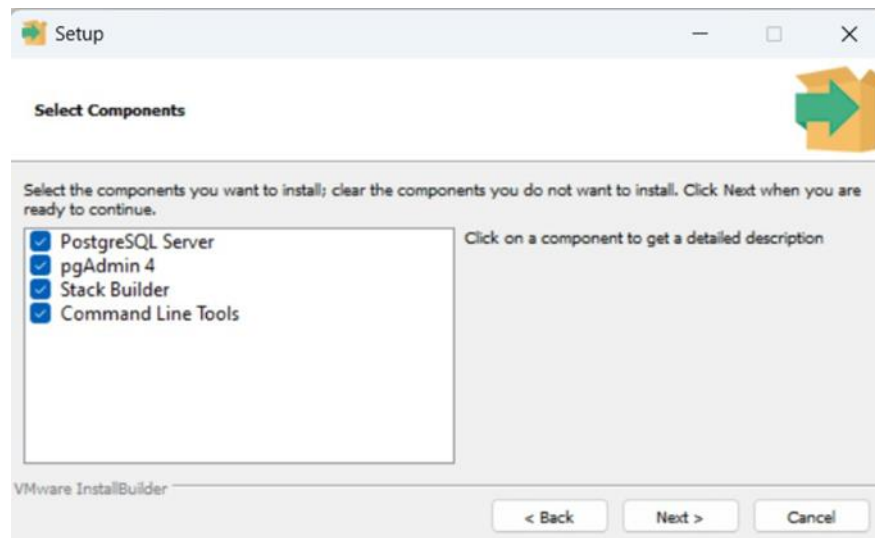


Рисунок 3 - Выбор компонентов, идущие вместе с PostgreSQL

- выбираете папку, где будут храниться базы данных;
- придумайте пароль для суперпользователя (ОБЯЗАТЕЛЬНО ЗАПОМНИТЕ ЕГО)!!! (рисунок 4);

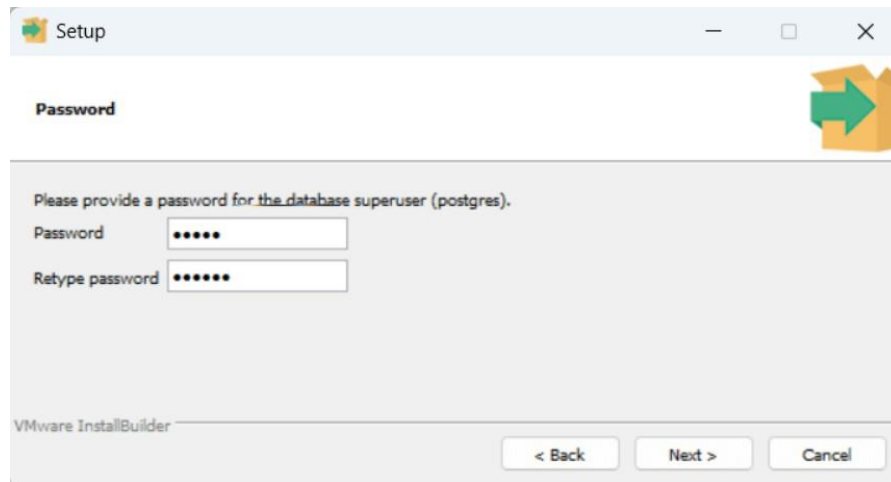


Рисунок 4 - Пароль для суперпользователя PostgreSQL

- оставляете порт по умолчанию (5432);
- в продвинутых настройках ничего не делаем и идем дальше;
- ждем, пока СУБД установится;
- поздравляю! Вы установили PostgreSQL (рисунок 5).

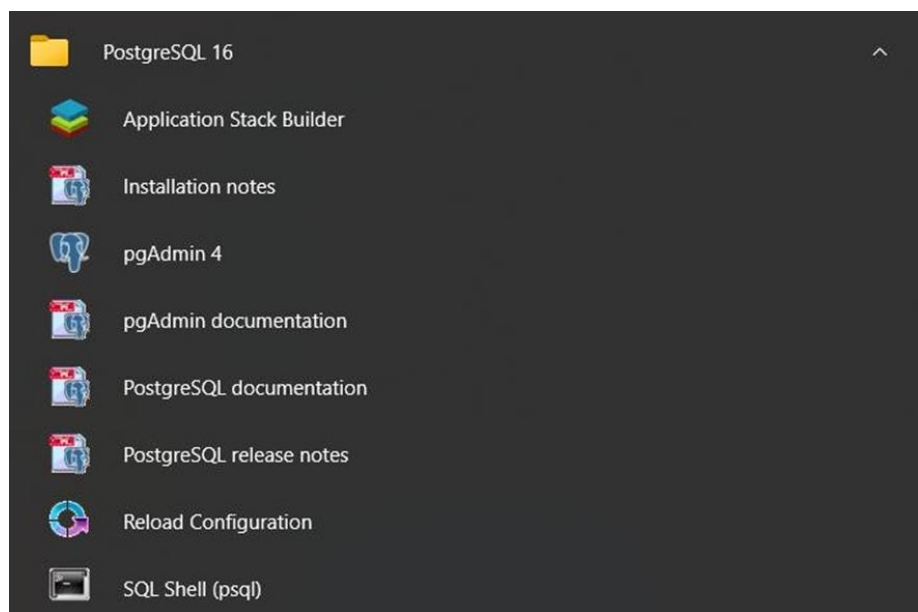


Рисунок 5 - Установленный PostgreSQL

pgAdmin – графический клиент для работы с PostgreSQL. Через него Вы можете создавать, редактировать, удалять, управлять базами данных в удобном виде. При входе в него, Вас попросят ввести пароль суперпользователя, здесь вводим тот пароль, который использовали при установке.

Для создания базы данных, сделаем действия как на скриншоте ниже (рисунок 6):

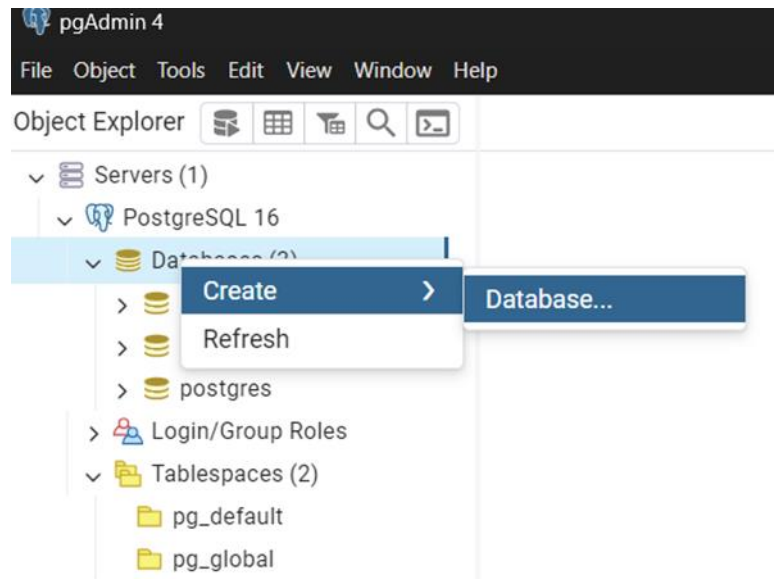


Рисунок 6 - Создание БД

После успешного создания БД нам напишет, что база данных успешно подключилась (рисунок 7).

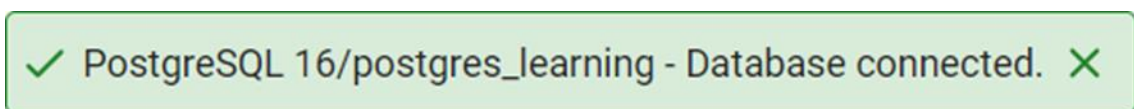


Рисунок 7 - Успешное соединение с БД

Для работы с запросами в PostgreSQL используется Query Tool (рисунок 8):

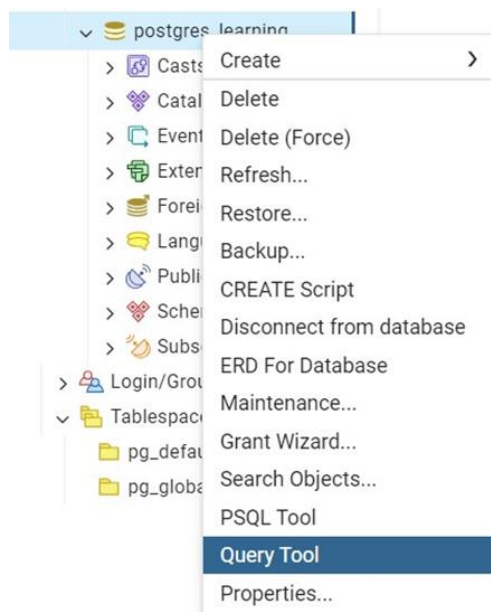


Рисунок 8 - Query tool

Вернемся к Django. Откроем файл настроек settings.py и изменим подключение с SQLite на PostgreSQL (рисунок 9).

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'postgres_learning',
        'USER': 'postgres',
        'PASSWORD': 'mypassword',
        'HOST': 'localhost',
    }
}
```

Рисунок 9 - Настройка DATABASES в settings.py для PostgreSQL

ENGINE – движок БД.

NAME – имя базы данных.

USER – имя пользователя.

PASSWORD – пароль пользователя.

HOST – хост базы данных.

Далее, установим psycopg2 для работы с PostgreSQL внутри Django (рисунок 10).

```
(.env) PS E:\Programmer\Django Projects\Pr10\postgresql_learning> pip install psycopg2
Collecting psycopg2
  Using cached psycopg2-2.9.9-cp311-cp311-win_amd64.whl.metadata (4.5 kB)
Using cached psycopg2-2.9.9-cp311-cp311-win_amd64.whl (1.2 MB)
Installing collected packages: psycopg2
Successfully installed psycopg2-2.9.9

[notice] A new release of pip is available: 24.0 -> 24.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.env) PS E:\Programmer\Django Projects\Pr10\postgresql_learning>
```

Рисунок 10 - Установка psycopg2

После этого необходимо выполнить миграции и убедиться, что мы все сделали правильно (рисунок 11).

```
PS E:\Programmer\Django Projects\Pr10\postgresql_learning> python.exe .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001 initial... OK
  Applying auth.0001 initial... OK
  Applying admin.0001 initial... OK
  Applying admin.0002 logentry_remove_auto_add... OK
  Applying admin.0003 logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001 initial... OK
```

Рисунок 11 - Выполнение миграций

Перейдем в pgAdmin и рассмотрим таблицы, которые добавились нам при развертывании проекта. Для этого, сначала нужно обновить БД (Refresh).

В остальном, вся работа она идентична, модели создаются также, миграции применяются также, все делается точно также. Но теперь, наш проект использует не SQLite, а PostgreSQL (рисунок 12-13).

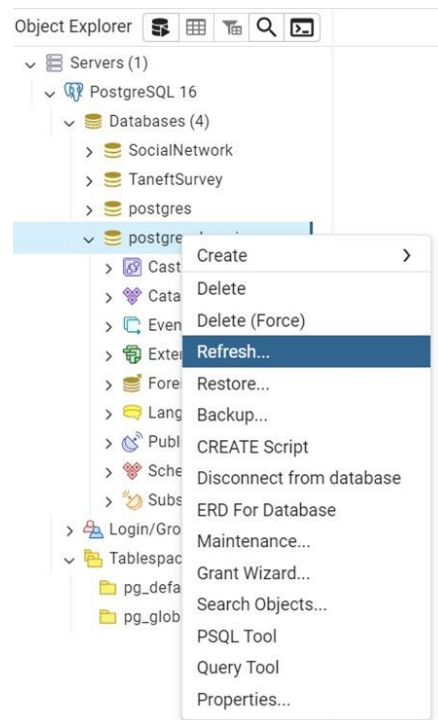


Рисунок 12 - Обновление базы данных

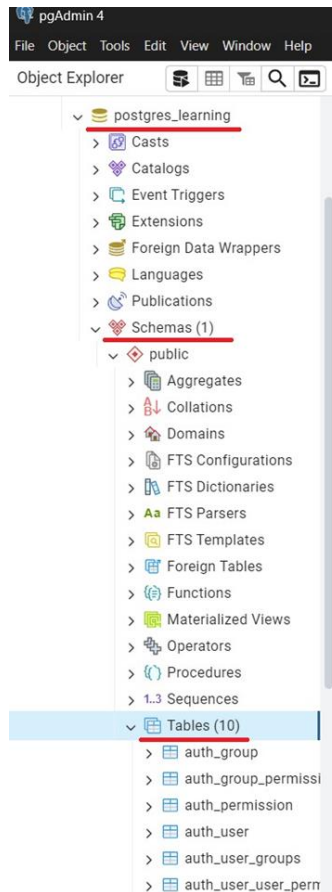


Рисунок 13 - Просмотр таблиц

Мы видим, что у нас создалось 10 новых таблиц в нашу БД (рисунок 14).

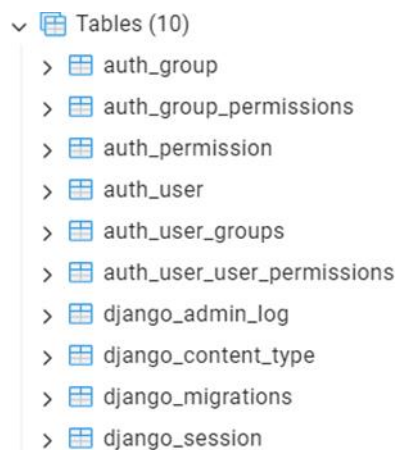


Рисунок 14 - Таблицы, добавленные в базу данных

4.2 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` подключите PostgreSQL. Создайте модель `Post` с полями «Заголовок», «Текст публикации» и «Дата создания». Реализуйте сохранение записей в базе данных и их отображение на странице списка публикаций.

Решение задачи:

1 Настройка PostgreSQL (settings.py)

В файле settings.py замените DATABASES:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'blog_db',
        'USER': 'postgres',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

2 Установка драйвера PostgreSQL

```
pip install psycopg2-binary
```

3 Модель Post (models.py)

В приложении posts/models.py:

```
from django.db import models
class Post(models.Model):
    title = models.CharField("Заголовок", max_length=255)
    content = models.TextField("Текст публикации")
    created_at = models.DateTimeField("Дата создания", auto_now_add=True)
    def __str__(self):
        return self.title
```

4 Миграции

```
python manage.py makemigrations
```

```
python manage.py migrate
```

5 Форма добавления (forms.py)

```
from django import forms
from .models import Post
class PostForm(forms.ModelForm):
```

```
    class Meta:
        model = Post
        fields = ['title', 'content']
```

6 Представления (views.py)

```
from django.shortcuts import render, redirect
```

```
from .models import Post
```

```
from .forms import PostForm
```

```
def post_list(request):
    posts = Post.objects.all().order_by('-created_at')
    return render(request, 'posts/post_list.html', {'posts': posts})
def create_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST)
        if form.is_valid():
```

```

        form.save()
        return redirect('post_list')
    else:
        form = PostForm()
        return render(request, 'posts/create_post.html', {'form': form})
7 URLs (posts/urls.py)
from django.urls import path
from . import views
urlpatterns = [
    path("", views.post_list, name='post_list'),
    path('create/', views.create_post, name='create_post'),
]
Подключение в главном urls.py:
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('posts/', include('posts.urls')),
]
8 Шаблон списка публикаций (post_list.html)
<h1>Список публикаций</h1>
<a href="{% url 'create_post' %}">Добавить пост</a>
{% for post in posts %}
    <div>
        <h2>{{ post.title }}</h2>
        <p>{{ post.content }}</p>
        <small>{{ post.created_at }}</small>
    </div>
    <hr>
{% endfor %}
9 Шаблон создания поста (create_post.html)
<h1>Создать публикацию</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Сохранить</button>
</form>

```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте `library_site` и приложении `users` настройте подключение к базе данных PostgreSQL в файле `settings.py`. Создайте модель

пользователя-профиля и выполните миграции для создания таблицы в базе данных.

2 В ранее созданном проекте `student_portal` и приложении `accounts` настройте подключение к PostgreSQL. Создайте модель `Student` с полями «Имя», «Фамилия», «Email» и сохраните данные студентов в базе данных.

3 В ранее созданном проекте `movie_portal` и приложении `profiles` подключите PostgreSQL. Создайте модель `MovieProfile` и реализуйте добавление записей в базу данных через форму.

4 В ранее созданном проекте `music_library` и приложении `accounts` настройте PostgreSQL. Создайте модель `Song` с полями «Название», «Исполнитель» и сохраните данные о песнях в базе данных.

5 В ранее созданном проекте `online_shop` и приложении `users` подключите PostgreSQL. Создайте модель `ProductUserProfile` и реализуйте сохранение данных пользователей в таблицу PostgreSQL.

6 В ранее созданном проекте `news_portal` и приложении `accounts` настройте PostgreSQL. Создайте модель `Article` с полями «Заголовок» и «Текст» и сохраните статьи в базе данных.

7 В ранее созданном проекте `tourism_portal` и приложении `tourists` подключите PostgreSQL. Создайте модель `Tourist` и реализуйте сохранение данных о туристах в базе данных PostgreSQL.

8 В ранее созданном проекте `sport_club_site` и приложении `members` настройте PostgreSQL. Создайте модель `Member` и сохраните данные участников клуба в базе данных через Django ORM.

9 В ранее созданном проекте `school_portal` и приложении `teachers` подключите PostgreSQL. Создайте модель `Teacher` и реализуйте сохранение данных преподавателей в базе данных.

10 В ранее созданном проекте `hospital_system` и приложении `patients` настройте PostgreSQL. Создайте модель `Patient` и реализуйте сохранение данных пациентов в базе данных PostgreSQL с использованием Django ORM.

11 В ранее созданном проекте `education_platform` и приложении `accounts` подключите PostgreSQL. Создайте модель `CourseProgress` и сохраните данные о прогрессе пользователя в базе данных.

12 В ранее созданном проекте `restaurant_site` и приложении `staff` настройте PostgreSQL. Создайте модель `Employee` и сохраните данные сотрудников ресторана в базе данных.

13 В ранее созданном проекте `car_catalog` и приложении `drivers` подключите PostgreSQL. Создайте модель `Driver` и реализуйте сохранение данных о водителях в базе данных PostgreSQL.

14 В ранее созданном проекте `portfolio_site` и приложении `main` настройте подключение к PostgreSQL. Создайте модель `Project` с полями «Название», «Описание», «Ссылка». Реализуйте сохранение проектов в базе данных и отображение списка через Django ORM.

15 В ранее созданном проекте `home_site` и приложении `home` подключите PostgreSQL. Создайте модель `UserRecord` с полями «Имя», «Email», «Пароль».

Реализуйте сохранение пользователей в базе данных PostgreSQL и вывод списка записей на отдельной странице.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

- 6.1 Тема лабораторной работы
- 6.2 Цель лабораторной работы
- 6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Как настроить подключение к базе данных PostgreSQL в проекте Django в языке программирования Python??

7.2 Какие параметры необходимо указать в DATABASES в файле settings.py в языке программирования Python??

7.3 Для чего используется библиотека psycopg2-binary в Django-проектах в языке программирования Python??

7.4 Что такое Django ORM и какую роль он играет при работе с базой данных в языке программирования Python??

7.5 Как происходит создание таблиц в PostgreSQL через Django в языке программирования Python??

7. 6 Что такое миграции в Django и для чего они используются в языке программирования Python??

Список используемых источников

1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.